

# SymPy Bug Report

## 1 Bug 57: missing Chi(x) logarithmic singularity at zero

### 1.1 Minimal Reproducer

```
from sympy import Chi, S, singularities, symbols

x = symbols("x")
s = singularities(Chi(x), x, S.Complexes)

print("actual_singularities =", s)
print("expected_contains_zero =", s.contains(0))
print("Chi(0) =", Chi(0))
```

### 1.2 Observed Behavior

```
actual_singularities = EmptySet
expected_contains_zero = False
Chi(0) = zoo
```

On the recorded SymPy checkout, the full reproducer also reported `right_limit = -oo` and a high-precision numerical sample near  $-45.47$  for  $\text{Chi}(10^{-20})$ . Thus `singularities` reports that there are no complex singularities, even though SymPy's own evaluation of  $\text{Chi}(0)$  is complex infinity.

### 1.3 Expected Behavior

The result should contain 0. For the minimal case, an appropriate result is  $\{0\}$ , or any set representation whose membership test confirms

$$0 \in \text{singularities}(\text{Chi}(x), x, \mathbb{C}).$$

### 1.4 Mathematical Explanation

The hyperbolic cosine integral has the local expansion

$$\text{Chi}(x) = \gamma + \log(x) + \int_0^x \frac{\cosh(t) - 1}{t} dt$$

near  $x = 0$ , equivalently

$$\text{Chi}(x) = \gamma + \log(x) + \frac{x^2}{2 \cdot 2!} + \frac{x^4}{4 \cdot 4!} + \cdots$$

The series after the logarithm is analytic at the origin, but  $\log(x)$  is singular there. Along the positive real axis,  $\text{Chi}(x) \rightarrow -\infty$  as  $x \rightarrow 0^+$ , and in the complex domain the logarithm gives a genuine logarithmic singularity at 0. This is not a harmless alternative representation or branch convention: a singularity set that is empty omits the point at which the function is not finite and at which SymPy itself evaluates  $\text{Chi}(0)$  to `zoo`.

## 1.5 Source-Level Diagnosis

The diagnosis locates the relevant limitation in `sympy/calculus/singularities.py`, in the implementation of `singularities` around lines 97–107. The traced public call path is the direct call to `sympy.calculus.singularities.singularities`: the expression is sympified, a small set of reciprocal trigonometric and hyperbolic functions is rewritten, and then the routine scans only a few syntactic patterns.

```
for i in e.atoms(Pow): ...
for i in expression.atoms(log, asech, acsch): ...
for i in expression.atoms(atanh, acoth): ...
```

An unevaluated  $\text{Chi}(x)$  expression has no `Pow`, `log`, `asech`, `acsch`, `atanh`, or `acoth` atoms, so none of the scans adds 0 to the singularity set. The diagnosis also notes that the missing information exists elsewhere in the SymPy source: `Chi` has `_atzero = S.ComplexInfinity` in `sympy/functions/special/error_functions.py`, and its `_eval_as_leading_term` method returns a leading term of the form  $\log(x) + \text{EulerGamma}$  near zero. The current `singularities` routine appears not to consult those function-level hooks, so the logarithmic singularity remains invisible to its syntactic detector. A plausible fix direction identified by the diagnosis is to add a special-function singularity hook, or otherwise make `singularities` consult function-level information such as values at candidate points or leading terms; for  $\text{Chi}(\arg)$ , roots of  $\arg$  should be included as logarithmic singularities.

## 1.6 Additional Failing Instances

The independent verification checks the shifted family

$$\text{singularities}(\text{Chi}(x - a), x, \mathbb{C})$$

for  $a = 0, 1, 2, 3, 4, 5$ . In every case, the expected singularity is the root of the inner argument,  $x = a$ , because the same local logarithmic behavior occurs when  $x - a \rightarrow 0$ . The verification also samples  $\text{Chi}(\epsilon)$  at high precision for a small positive  $\epsilon$ , confirming the downward divergence on the real approach to the singular point.

| Input / Expression                                                                   | SymPy behavior                                     | Expected behavior |
|--------------------------------------------------------------------------------------|----------------------------------------------------|-------------------|
| $\text{Chi}(x)$ in <code>singularities(<math>\cdot, x, \mathbb{C}</math>)</code>     | Does not contain 0; recorded result is $\emptyset$ | Contains 0        |
| $\text{Chi}(x - 1)$ in <code>singularities(<math>\cdot, x, \mathbb{C}</math>)</code> | Does not contain 1                                 | Contains 1        |
| $\text{Chi}(x - 2)$ in <code>singularities(<math>\cdot, x, \mathbb{C}</math>)</code> | Does not contain 2                                 | Contains 2        |
| $\text{Chi}(x - 3)$ in <code>singularities(<math>\cdot, x, \mathbb{C}</math>)</code> | Does not contain 3                                 | Contains 3        |
| $\text{Chi}(x - 4)$ in <code>singularities(<math>\cdot, x, \mathbb{C}</math>)</code> | Does not contain 4                                 | Contains 4        |
| $\text{Chi}(x - 5)$ in <code>singularities(<math>\cdot, x, \mathbb{C}</math>)</code> | Does not contain 5                                 | Contains 5        |

## 1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public matches were [sympy/sympy#18455](#), a broad plotting and singularity-detection discussion noting limitations for non-rational functions, and [sympy/sympy#26208](#), which concerns asymptotic series expansions for Shi and Chi at infinity. The deduplication verdict is that this is a new instance of a known failure mode: the general weakness for non-rational and special-function singularities is known, but no exact public report was identified for singularities( $\text{Chi}(x), x, \mathbb{C}$ ) returning  $\emptyset$  or for missing the Chi singularity at zero. The case remains individually actionable as a concrete regression test and a specific special-function hook target.

## 1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Chi, S, singularities, symbols

@pytest.mark.parametrize("a", [0, 1, 2, 3, 4, 5])
def test_singularities_chi_includes_logarithmic_origin(a):
    x = symbols("x")
    result = singularities(Chi(x - a), x, S.Complexes)
    assert result.contains(S(a)) is S.true
```